

Investigación REACT:

➤ ¿Qué son los Hooks de Rendimiento? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Los Hooks de rendimiento en React son funciones integradas diseñadas para evitar re-renderizados innecesarios y cálculos redundantes entre renders de un componente. En React, cuando el estado o las propiedades (props) de un componente cambian, toda la función del componente se vuelve a ejecutar de arriba abajo, sirven para indicarle a React qué valores o funciones deben preservarse en memoria mientras sus dependencias no cambien.

Características:

- Reciben un arreglo de dependencias y solo recalculan el valor o recrean la función si al menos una dependencia cambia por comparación superficial.
- Guardan referencias y comparan arreglos de dependencias, lo cual consume memoria y CPU; usarlos en operaciones triviales empeora el rendimiento en lugar de mejorarlo.
- React no garantiza que un valor memorizado viva indefinidamente (puede liberarlo si necesita memoria), por lo que el código debe funcionar correctamente sin ellos y tratarlos solo como una optimización.

Ejemplo:

```
import React, { useState, useMemo, useCallback } from 'react';

// Componente hijo optimizado con React.memo.
// Solo se re-renderiza si sus props (onReset) cambian por referencia.
const BotonReinicio = React.memo(({ onReset }) => {
  console.log('-> Render de BotonReinicio');
  return <button onClick={onReset}>Reiniciar Contador</button>;
});

export default function ContadorAvanzado() {
  const [contador, setContador] = useState(0);
```

```
const [temaOscuro, setTemaOscuro] = useState(false);
```

```
// 1. useMemo: Evita re-ejecutar el cálculo pesado al alternar el tema
```

```
const resultadoPesado = useMemo(() => {  
  console.log('-> Ejecutando cálculo pesado...');  
  let total = 0;  
  for (let i = 0; i < 50_000_000; i++) {  
    total += 1;  
  }  
  return contador * 2;  
}, [contador]); // Solo se recalcula si 'contador' cambia
```

```
// 2. useCallback: Mantiene la misma referencia de la función
```

```
// Evita que BotonReinicio se renderice de nuevo cuando cambia el tema
```

```
const reiniciar = useCallback(() => {  
  setContador(0);  
}, []); // Sin dependencias: la referencia nunca cambia
```

```
const estilo = {  
  backgroundColor: temaOscuro ? '#333' : '#fff',  
  color: temaOscuro ? '#fff' : '#000',  
  padding: '20px',  
  borderRadius: '8px'  
};
```

```
return (  
  <div style={estilo}>  
    <h2>Contador: {contador}</h2>
```

```
<p>Resultado del cálculo: {resultadoPesado}</p>
```

```
<button onClick={() => setContador(c => c + 1)}>Incrementar</button>
```

```
<button onClick={() => setTemaOscuro(t => !t)}>Cambiar Tema</button>
```

```
<div style={{ marginTop: '10px' }}>
```

```
  <BotonReinicio onReset={reiniciar} />
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```

Si el ciclo de 50 millones de iteraciones estuviera suelto dentro del componente, al hacer clic en "Cambiar Tema" (setTemaOscuro), todo el componente se ejecutaría de nuevo y la interfaz se congelaría brevemente para repetir el bucle innecesariamente. Con useMemo, como la dependencia [contador] no cambió, React devuelve el resultado que ya tenía en memoria al instante.

➤ ¿Qué es el contexto en React? ¿Cuáles son sus características y para qué sirve? ¿Qué diferencia existen con los props? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

El Contexto es una API nativa de React diseñada para compartir datos que pueden considerarse globales o compartidos a lo largo de un árbol de componentes, sin necesidad de pasar explícitamente esos datos como props manualmente por cada nivel intermediario. Sirve principalmente para resolver el problema conocido como Prop Drilling (la necesidad de pasar datos a través de componentes intermedios que no los necesitan, solo para que lleguen a un hijo situado en un nivel inferior).

Características:

- Utiliza un componente proveedor que suministra el valor, y los componentes anidados lo leen mediante el hook useContext.
- No es necesariamente un estado global único para toda la app; un contexto puede envolver solo una sección de la jerarquía
- Cualquier componente que consuma el contexto se volverá a renderizar automáticamente cuando el valor provisto en value cambie.

Se diferencia con las props debido a que las props pasan datos de forma directa y manual de un componente padre a su hijo inmediato, mientras que el context difunde datos globalmente a través del árbol de componentes sin necesidad de intermediarios.

Ejemplo:

```
import React, { createContext, useContext, useState } from 'react';
```

```
// 1. Crear el Contexto
```

```
const TemaContext = createContext();
```

```
// 2. Componente Proveedor (encapsula estado y lógica)
```

```
export function ProveedorTema({ children }) {
```

```
  const [tema, setTema] = useState('claro');
```

```
  const alternarTema = () => {
```

```
    setTema((prev) => (prev === 'claro' ? 'oscuro' : 'claro'));
```

```
  };
```

```
  return (
```

```
    <TemaContext.Provider value={{ tema, alternarTema }}>
```

```
      {children}
```

```
    </TemaContext.Provider>
```

```
  );
```

```
}
```

```
// 3. Componente intermedio que NO recibe props relacionadas al tema
```

```
function BarraNavegacion() {
```

```
  console.log('-> Render de BarraNavegacion (sin prop drilling)');
```

```
  return (
```

```

<nav style={{ padding: '10px', borderBottom: '1px solid #ccc' }}>

  <span>Mi Aplicación</span>

  { /* BotonTema está anidado pero no requiere que BarraNavegacion le pase
nada */}

  <BotonTema />

</nav>

);
}

```

// 4. Componente consumidor que accede directamente al Contexto

```

function BotonTema() {

  const { tema, alternarTema } = useContext(TemaContext);

  const estilos = {

    background: tema === 'claro' ? '#eee' : '#333',

    color: tema === 'claro' ? '#000' : '#fff',

    border: '1px solid #999',

    padding: '6px 12px',

    marginLeft: '15px',

    cursor: 'pointer',

    borderRadius: '4px'

  };

  return (

    <button style={estilos} onClick={alternarTema}>

      Modo actual: {tema} (Click para cambiar)

    </button>

  );
}

```

```
}
```

```
// 5. Componente Principal
```

```
export default function App() {
```

```
  return (
```

```
    <ProveedorTema>
```

```
      <div style={{ fontFamily: 'sans-serif' }}>
```

```
        <BarraNavegacion />
```

```
        <main style={{ padding: '20px' }}>
```

```
          <h3>Bienvenido</h3>
```

```
          <p>El botón de navegación consume el contexto sin pasar props  
intermedias.</p>
```

```
        </main>
```

```
      </div>
```

```
    </ProveedorTema>
```

```
  );
```

```
}
```

El componente proveedor (ProveedorTema) almacena el estado del tema y lo distribuye globalmente a través de TemaContext, permitiendo que el botón hijo (BotonTema) lea y modifique ese valor mediante useContext sin necesidad de transmitir propiedades a través del componente intermedio (BarraNavegacion).